

Natural Computing SoSe25

Exercise Sheet 5 (Solution) — July 3rd, 2025

Thomas Gabor, Maximilian Zorn

1 Optimization 101

(i) To deepen your understanding of the optimization concepts introduced in the lecture, define $\mathcal{X}$, $\mathcal{T}$, and τ , as well as an example for $x \in \mathcal{X}$, for the following example problems. Also state if τ is to be *maximized* or *minimized*.

Recall the following example based on the lecture:

Knapsack Problem: *Maximize the combined value of stored items $o \in \{A, \dots, E\}$ with some $\text{value}(o)$ and some $\text{weight}(o)$ within some given total weight capacity M .*

$$\mathcal{X} = \{x \mid x \subseteq \{A, \dots, E\} \wedge \sum_{o \in x} \text{weight}(o) \leq M\}$$

$$x =_{\text{(e.g.)}} \{A, B, E\}$$

$$\mathcal{T} = \mathbb{N}$$

$$\text{max. } \tau = x \mapsto \sum_{o \in x} \text{value}(o)$$

(a) Traveling Salesperson Problem (TSP): *Given V , which is a set of $|V| = n$ destinations, find the fastest route for the salesperson so that the distance they have to travel between all destinations (e.g., $D(i, j)$ between destinations $i, j \in V$) is minimal. They have to visit every destination exactly once and need to end up in the destination they started at.*

$$\begin{aligned}\mathcal{X} &= \{x \mid x \in V^n \wedge \forall i, j \in [0; n) \subset \mathbb{N} : i \neq j \implies x_i \neq x_j\} \\ x &=_{(\text{e.g.})} (A, C, B) \text{ for } V = \{A, B, C\} \\ \mathcal{T} &= \mathbb{R}\end{aligned}$$

$$\min. \tau = x \mapsto D(x_{n-1}, x_0) + \sum_{i=0}^{n-2} D(x_i, x_{i+1})$$

We could also decide on an alternative formulation like this:

$$\begin{aligned}\mathcal{X} &= \{x \mid x \in V^{n+1} \wedge x_0 = x_n \wedge \forall v \in V : \exists i : x_i = v\} \\ x &=_{(\text{e.g.})} (A, C, B, A) \text{ for } V = \{A, B, C\} \\ \mathcal{T} &= \mathbb{R}\end{aligned}$$

$$\min. \tau = x \mapsto \sum_{i=0}^{n-1} D(x_i, x_{i+1})$$

(b) Longest common beginning substring problem: For a set of words S , e.g., $S = \{\text{"AACBAB"}, \text{"CABBAA"}\}$, with $S \subset \{ \text{"A"}, \text{"B"}, \text{"C"} \}^* = \{ \langle c_i \rangle_{1 \leq i \leq N} \mid c_i \in \{ \text{"A"}, \text{"B"}, \text{"C"} \} \wedge N \in \mathbb{N} \}$ find the longest string of characters that appears at the start of all words of S .

$$\begin{aligned}\mathcal{X} &= \{ \text{"A"}, \text{"B"}, \text{"C"} \}^* \\ x &=_{(\text{e.g.})} \text{"AA"} = \langle \text{"A"}, \text{"A"} \rangle \\ \mathcal{T} &= \mathbb{N}\end{aligned}$$

$$\max. \tau = x \mapsto \tau^\dagger(x, 1)$$

$$\text{where } \tau^\dagger = (\langle c_i \rangle_{1 \leq i \leq L}, d) \mapsto \begin{cases} 1 + \tau^\dagger(\langle c_i \rangle_{2 \leq i \leq L}, d+1) & \text{if for all } s \in S \text{ it holds that } s_d = c_1 \\ 0 & \text{otherwise} \end{cases}$$

We could also decide on an alternative formulation for τ without recursion:

$$\max. \tau = \langle c_i \rangle_{1 \leq i \leq L} \mapsto \sum_{l=1}^m \mathbb{I}[\forall s \in S : \forall k \leq l : c_k = s_k]$$

where $m = \min_{s \in S} |s|$ and $\mathbb{I}$ is the indicator function.

(c) (Bonus:) Longest common ~~beginning~~ substring problem: For a set of words S as given above, find the longest string of characters that appears somewhere within all words of S .

$$\begin{aligned}
\mathcal{X} &= \{\text{"A"}, \text{"B"}, \text{"C"}\}^* \\
x &=_{\text{(e.g.)}} \text{"AA"} = \langle \text{"A"}, \text{"A"} \rangle \\
\mathcal{T} &= \mathbb{N} \\
\min. \tau &= x \mapsto - \max_{start_in \in (S \rightarrow \{1, \dots, m\})} \tau^\otimes(x, start_in) \\
\text{where } m &= \min_{s \in S} |s| \\
\text{and } \tau^\otimes &= (\langle c_i \rangle_{1 \leq i \leq L}, start_in) \mapsto \\
&\quad \begin{cases} 1 + \tau^\otimes(\langle c_i \rangle_{2 \leq i \leq L}, (s \mapsto start_in(s) + 1)) & \text{if for all } s \in S \text{ it holds that } s_{start_in(s)} = c_1 \\ 0 & \text{otherwise} \end{cases}
\end{aligned}$$

We could also decide on another alternative formulation for τ without recursion:

$$\max. \tau = \langle c_i \rangle_{1 \leq i \leq L} \mapsto \max_{start_in \in (S \rightarrow \{1, \dots, m\})} \max_{l \in \{1, \dots, m\}} l \cdot \mathbb{I}[\forall s \in S : \forall k \leq l : c_k = s_{start_in(s)+k}]$$

where $m = \min_{s \in S} |s|$ and $\mathbb{I}$ is the indicator function.

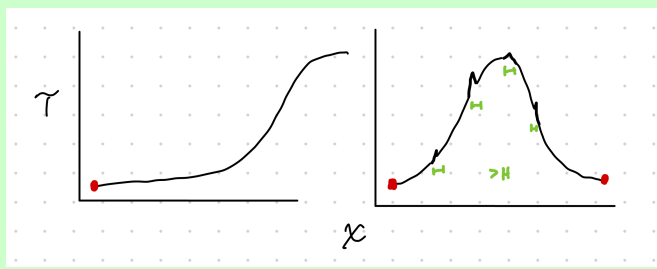
(d) (Bonus:) Chess: We search for a strategy for a black player in a game of chess against a deterministic white player where the player tries to win by minimizing the total value of the opponent's chess pieces (by taking them). A strategy is (in this case) a series of valid chess moves. Let $\mathcal{M}$ be the set of all possible (valid or invalid) chess moves. When B is the set of all board configurations and $b_0 \in B$ is the starting configuration, let $M(b)$ for $b \in B$ be the set of all valid moves for black given the board b . For $\mu \in M(b)$, let $\mu(b) \in B$ be the resulting board configuration when playing the move μ . Let $w(b) \in B$ be the deterministic resulting board configuration when white makes a move on board b . Let $P = \text{white_pieces}(b)$ be a series of all remaining white pieces in a fixed order, i.e., $P = \langle P_j \rangle_{1 \leq j \leq |P|}$ with $P_j \in \{\text{"Pawn"}, \dots, \text{"King"}\}$ for all j . Pawns are considered worth one point, knights and bishops about three points each, rooks about five points (the value difference between a rook and a bishop or knight being known as the exchange), and queens about nine points. The king is more valuable than all of the other pieces combined, since its checkmate loses the game.

$$\begin{aligned}
\mathcal{X} &= \mathcal{M}^* = \{ \langle \mu_i \rangle_{1 \leq i \leq N} \mid \mu_i \in \mathcal{M} \wedge N \in \mathbb{N} \} \\
x &=_{(\text{e.g.})} \langle \text{"black pawn from G7 to G6"}, \\
&\quad \text{"black rook from A8 to A4"} \rangle \\
\mathcal{T} &= \mathbb{N} \\
\text{min. } \tau &= x \mapsto \tau^\dagger(x, b_0) \\
\text{where } P &= \text{white_pieces}(b) \\
\text{in } \tau^\dagger &= (\langle \mu_i \rangle_{1 \leq i \leq N}, b) \mapsto \begin{cases} \sum_{j=1}^{|P|} \text{value}(P_j) & \text{if } N = 0, \text{ i.e., } \langle \mu_i \rangle_{1 \leq i \leq N} = \langle \rangle, \\ \infty & \text{else if } \mu_1 \notin M(b), \\ \tau^\dagger(\langle \mu_i \rangle_{2 \leq i \leq N}, w(\mu_1(b))) & \text{else.} \end{cases} \\
\text{where } \text{value}(p) &= \begin{cases} 1 & \text{if } p = \text{"Pawn"}, \\ 3 & \text{if } p = \text{"Knight"} \text{ or } p = \text{"Bishop"}, \\ 5 & \text{if } p = \text{"Rook"}, \\ 9 & \text{if } p = \text{"Queen"}, \\ 40 & \text{if } p = \text{"King"}. \end{cases}
\end{aligned}$$

Note: This is not a viable way to model a chess AI!

(ii) In the lecture we have also introduced the notions of *working to some extent* and *improving*. For each of the following minimization scenarios, draw a 1D solution landscape of your choice, where ...

(a) an optimization run with the *stochastic hill climbing* algorithm (with a neighborhood of $x \pm N$) is (almost) certain to be improving after g steps and where the *end result* is always the *best result*.

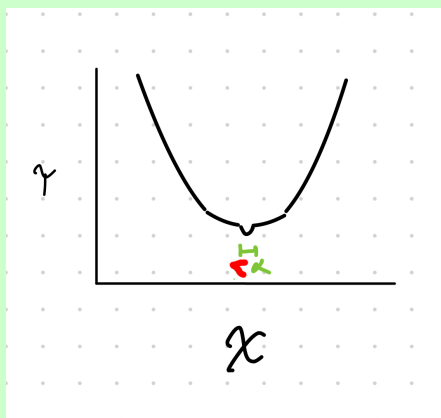


Examples include landscapes that only continuously improve or landscapes with local optima that are smaller than the neighborhood radius N (i.e., are not an obstacle with enough samples).

(b) an optimization run with the *stochastic hill climbing* algorithm (with a neighborhood of $x \pm N$) is (almost) certain to be improving after g steps and where the *end result* is (almost) never the *best result*.

Stochastic hill climbing will never allow for samples to be chosen that are worse than the current best (*greedy property*). As such every allowed sample is automatically the *best result* so far. Therefore there is no situation in which the *end result* is not the *best result*.

(c) an optimization run with the *gradient descent* algorithm is (almost) certain to be working to some extent after g steps and where the *end result* is (almost) never the *best result*.



(Stochastic) Gradient descent is also a greedy algorithm that always continues down the gradient, as such in *most cases* the *end result* is also the *best result*. The size of the update step, however, is scaled by the parameter α (often called the *learning rate*), which could result in the process ‘overshooting’ the optimum. If the optimum happens to be small enough (as is the case here) and the α parameter is too big and does not exactly hit the optimum, the *end result* will most likely not be the *best result*. (Therefore, in real applications the learning rate is often annealed towards very small values with a *learning rate schedule*.)

(iii) What are the main weaknesses of the *stochastic hill climbing* algorithm and the *gradient descent* algorithm?

- *stochastic hill climbing*: Landscapes with many big, local optima or better solutions out of reach of the neighborhood radius.
- *gradient descent*: Landscapes without gradient (e.g., plateaus or discrete $\mathcal{T}$), as well as global optima hidden behind or between local optima. And, as mentioned above, a badly chosen α parameter.

2 Bright Side Search

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *bright side search*.

Definition 1 (optimization (shortened)). *Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.*

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Algorithm 1 (bright side search). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = [0; 100] \subset \mathbb{N}$ and $\mathcal{T} = [0; 100] \subset \mathbb{N}$. The process $\mathcal{D}$ continues via bright side search iff e is given via

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} \lceil x_t/2 \rceil & \text{if } \tau(x_t - 1) < \tau(x_t + 1), \\ \lfloor x_t + (100 - x_t)/2 \rfloor & \text{otherwise.} \end{cases}$$

For $x \notin [0; 100]$ we assume that $\tau(x) = \infty$.

Note that $\lceil x \rceil$ is x rounded up to the nearest integer and $\lfloor x \rfloor$ is x rounded down to the nearest integer.

(a) Let the target function to be minimized be

$$\tau(x) = |x - 42|.$$

Let the initial state be $x_0 = 65$. **Execute bright side search (cf. Algorithm 1) in this setting for three time steps, i.e., compute x_1, x_2, x_3 . Briefly state why this run will never reach the global optimum at $x^* = 42$.**

step 1:

$$\begin{aligned} \tau(x_0 - 1) &= \tau(64) = 22 < \tau(x_0 + 1) = \tau(66) = 24 \\ \implies x_1 &= 33 \end{aligned}$$

step 2:

$$\begin{aligned} \tau(x_1 - 1) &= \tau(32) = 10 \geq \tau(x_1 + 1) = \tau(34) = 8 \\ \implies x_2 &= 66 \end{aligned}$$

step 3:

$$\begin{aligned} \tau(x_2 - 1) &= \tau(65) = 23 < \tau(x_2 + 1) = \tau(67) = 25 \\ \implies x_3 &= 33 \end{aligned}$$

With $x_1 = x_3$ we are already in a loop that will continue forever and does not contain the global optimum x^* .

(b) We say an optimization algorithm converges on the global optimum if (given enough time) it is likely to come closer to the global optimum than it was initially and if, once it reaches the global optimum, it stays close to that point.

Give a target function $\tau^\dagger : \mathcal{X} \rightarrow \mathcal{T}$ in accordance with Algorithm 1 with one single global optimum so that bright side search always reaches the global optimum. Give that global optimum and briefly state why bright side search can reach it. Then show that bright side search still does not converge on that global optimum.

$\tau'(x) = x$ with an optimum at $x^* = 0$ with $\tau(x^*) = 0$.

Bright side search can reach that because the target function is monotonic so it always travels down the slope in one direction (by halving its current sample).

Bright side search cannot converge because at $x_t = x^* = 0$ it then computes $x_{t+1} = 50$ and begins its journey once more.